00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D

ALGORİTMA VE PROGRAMLAMA – II

HAFTA 09

Yığın ve Kuyruk

Soyut Veri Tipleri ve Algoritmik Uygulamaları

MODÜL III · DİNAMİK VERİ VE HIZLI ERİŞİM

Dr. Öğr. Üyesi Mehmet Sevri · RTEÜ Bilgisayar Mühendisliği · 2026–2027 Bahar

14 HAFTALIK YOLCULUK

Yol haritasında neredeyiz?

H1 Doğruluk ve Ölçüm	**H2** Asimptotik Analiz	**H3** Özyineleme ve Geri İzleme	**H4** Arama Algoritmaları	**H5** Temel Sıralama	**H6** Böl ve Fethet	**H7** Öbek ve Doğrusal Sıralama
0x00	0x01	0x02	0x03	0x04	0x05	0x06
H8 Dinamik Dizi ve Bağlı Liste	**H9** **Yığın ve Kuyruk**	**H10** Karma Tabloları	**H11** Graflar: BFS ve DFS	**H12** Açgözlü Algoritmalar	**H13** Dinamik Programlama	**H14** Sentez ve Final Projesi
0x07	0x08	0x09	0x0A	0x0B	0x0C	0x0D

Vize: H1–H7 · Ara proje: H7 (teslim: H8 başı) · Final projesi: H14

TAHMİN ET

Geçen haftadan ve AP-1'den üç soru

- **S1 (H8).** Kapasitesi 1'den başlayıp dolunca iki katına çıkan dinamik diziye 20 eleman ekleniyor. Kaç kez büyür, toplam kaç eleman kopyalanır?
- **S2 (H8).** Yalnızca baş işaretçisi olan tek bağlı listenin sonuna 1 000 eleman tek tek ekleniyor. Toplam kaç düğüm ziyaret edilir? Son işaretçisiyle?
- **S3 (AP-1 H9).** `faktoriyel(4)` çalışırken çağrı yığınında en çok kaç çerçeve bulunur? İlk hangi çağrı döner?

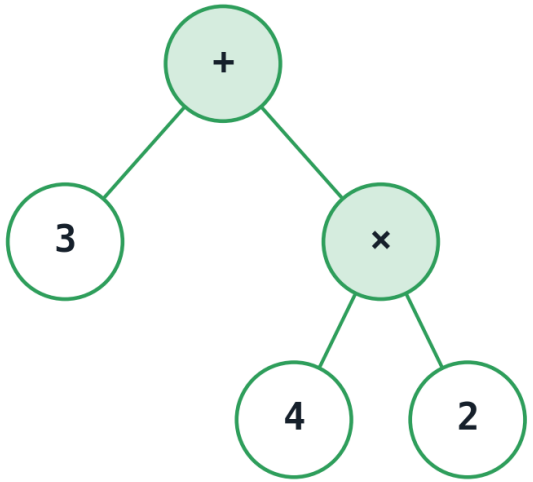
Yanıtlar: üç yapı bu haftanın üç gerçekleştirimi

- **S1:** kapasite 1 → 2 → 4 → 8 → 16 → 32: 5 büyütme, 1 + 2 + 4 + 8 + 16 = **31** kopya, ekleme başına 1,55 (10^6 eklemede 1,049): it amortize O(1).
- **S2:** n(n − 1)/2 = **499 500** ziyaret; son işaretçisiyle 0. Kuyruğun sonuna ekleme bu yüzden son işaretçisi ister.
- **S3:** dört çerçeve; ilk dönen en son çağrılan `faktoriyel(1)`: **son giren ilk çıkar**. Özyineleme örtük bir yığındır.
- Bu hafta: dizi tabanlı yığın (S1), baş + son işaretçili kuyruk (S2), çağrı yığınının açık yığınla yazılması (S3).

Büyük Soru: 3 + 4 × 2 neden 11, 14 değil?

öncelik kuralıyla: 3 + (4 × 2)

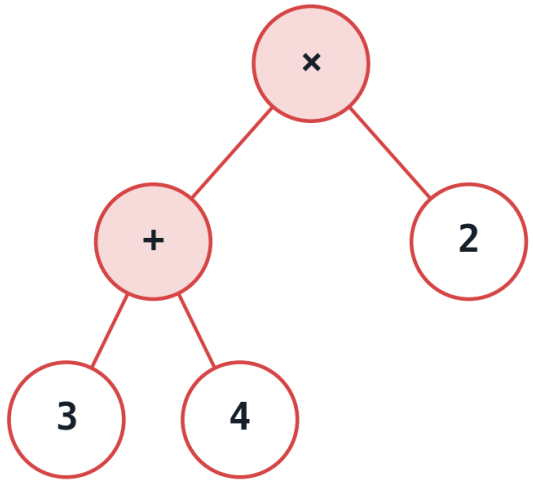
= 11



sonek: 3 4 2 * + önek: + 3 * 4 2

soldan sağa okununca: (3 + 4) × 2

= 14



sonek: 3 4 + 2 * önek: * + 3 4 2

araekte ikinci okuma için parantez gerekir; sonek ve önek gösterimde parantez ve öncelik kuralı yoktur

*İki adım, iki yığın: Örnek 5 3 + 4 * 2 -> 3 4 2 * + · Örnek 6 3 + 4 * 2 = 11*

GERÇEK DÜNYADAN

1957 · 1960 · 1961

Bir çevirmen, üç yönlü bir demiryolu kavşağı

- Dijkstra, X1 bilgisayarı için ALGOL 60 çevirmeni yazarken ifade çevirisini **MR 35/61** raporunda (Kasım 1961) anlattı: “shunting at a three way railroad junction”.
- İşlenenler doğrudan çıktıya; işleçler **translator stack**'te bekler. Öncelikler: + − 9; × / tekli eksi 10; ↑ 11.
- Kural “greater then or equal to” (sic): bütün işleçler **soldan** birleşir; ALGOL 60'ta 2↑n↑k = (2↑n)↑k (Backus vd., 1963). Bu derste ^ **sağdan** birleşir.
- Aynı yıllarda Bauer ve Samelson formül çevirisini **iki yığınla** yaptı: “numbers cellar” ve “operations cellar” (ABD patenti 1962; makale 1960).

Dijkstra, E. W. (1961). MR 35/61, Mathematisch Centrum · Bauer & Samelson (1962), U.S. Patent 3,047,228 · Samelson & Bauer (1960), CACM 3(2)

1972: HP-35 — RPN ve cebe sığan dört katlı yığın

HP-35: (3 × 4) + (5 × 6) tuş dizisi ve dört yazmaçlı işlem yığını (X tepe)

	3	ENTER↑	4	×	5	ENTER↑	6	×	+
T	–	–	–	–	–	–	–	–	–
Z	–	–	–	–	–	12	12	–	–
Y	–	3	3	–	12	5	5	12	–
X	**3**	**3**	**4**	**12**	**5**	**5**	**6**	**30**	**42**

ENTER ↑ X'i Y'ye kopyalar (y → Z, z → T, t kaybolur); × ve + Y ile X'i birleştirir, yığın düşer

(3 × 4) + (5 × 6) tuş dizisi: X, Y, Z, T yazmaçları bir "operational stack"; parantez tuşu gerekmez (Whitney vd., 1972)

ÖĞRENME ÇIKTILARI

Bu hafta sonunda neler yapabileceksiniz?

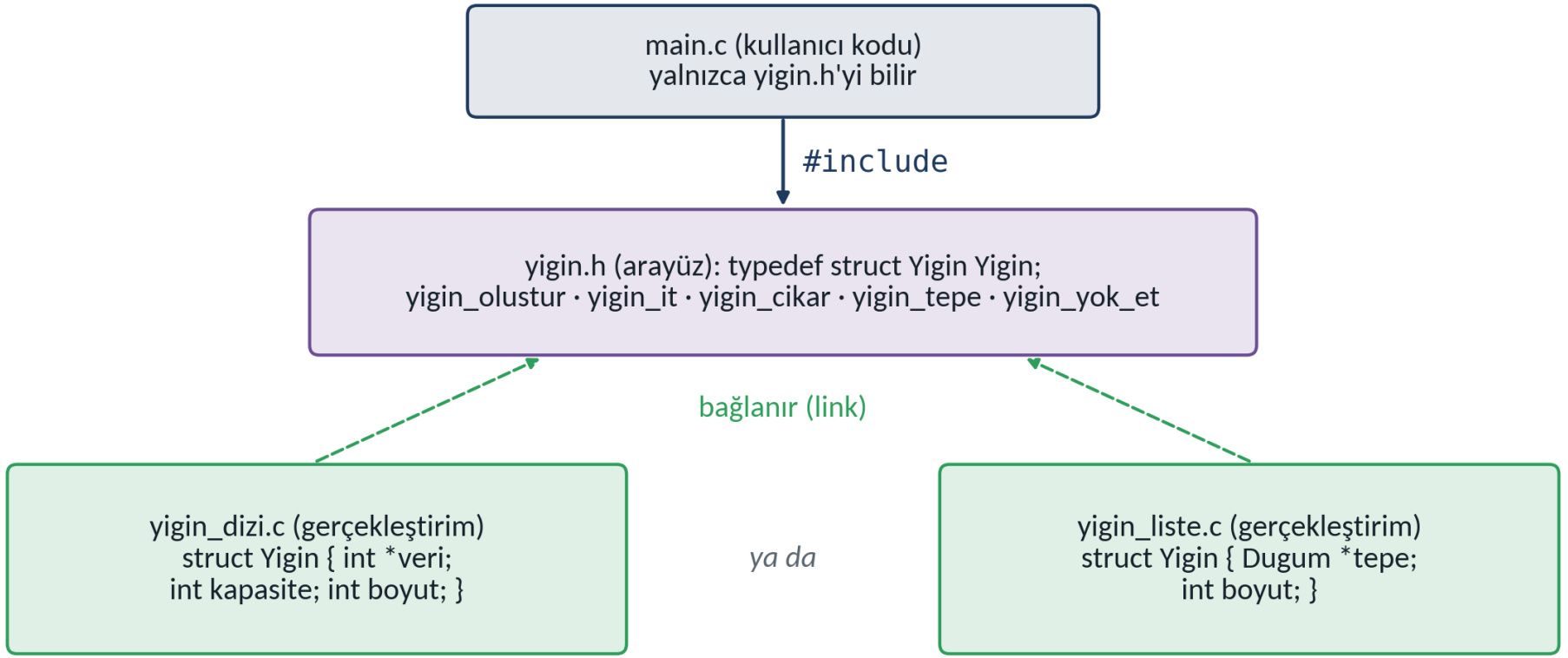
ÖÇ 9.1	Soyut veri tipi kavramını arayüz–gerçekleştirim ayrımıyla açıklar; yığın ve kuyruğu dizi ve bağlı listeyle gerçekler.
ÖÇ 9.2	Dairesel tamponla sabit kapasiteli kuyruğu, dolu ve boş durumlarını doğru ayırt ederek gerçekler.
ÖÇ 9.3	Yığın tabanlı algoritmalarla parantez denetimi, sonek ifade değerlendirme ve araek → sonek dönüşümü (Shunting-yard) yapar.
ÖÇ 9.4	Monotonik yığınla “sonraki büyük eleman” türü problemleri O(n) sürede çözer.
ÖÇ 9.5	Özyinelemeli bir algoritmayı açık yığın kullanan yinelemeli biçime dönüştürür; kuyruğu simülasyon ve katman katman işlemede kullanır.

Soyut veri tipi: işlemler ve sözleşmeler

ADT bir değerler kümesi, işlemler ve her işlemin **sözleşmesidir** (önkoşul, sonkoşul, hata bildirimi); verinin bellekte nasıl tutulduğunu söylemez.

İşlem	Yığın (LIFO)	Kuyruk (FIFO)	Sözleşme (bu haftanın C arayüzü)
ekle	`yigin_it(y, x): tepeye`	`kuyruk_ekle(q, x): sona`	O(1); doluysa taşma: false, yapı değişmez
çıkar	`yigin_cikar(y, &x): tepedeki`	`kuyruk_cikar(q, &x): baştaki`	O(1); boşsa alt taşma: false, *x değişmez
bak	`yigin_tepe(y, &x)`	`baştaki (deque_on)`	O(1); boş yapıda false
oluştur, yok et	`yigin_olustur / yigin_yok_et`	`kuyruk_olustur / kuyruk_yok_et`	oluşturan yok eder (sahiplik)

C'de opak tip: arayüz başlıkta, gerçekleştirim .c dosyasında



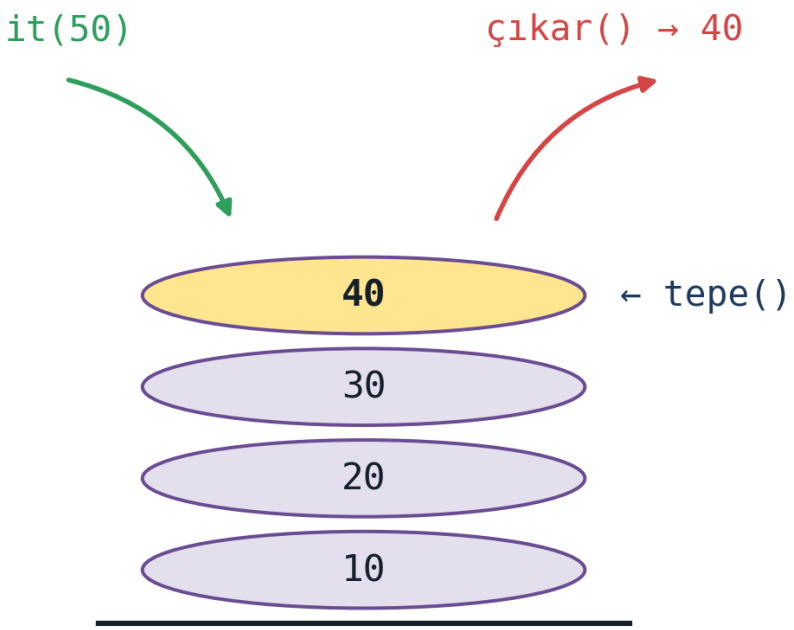
aynı main.c iki gerçekleştirimle derlendi → cmp: fark yok (iki gerçekleştirimin stdout'u bayt bayt aynı)

opak_ihlal.c alanlara erişince derleyici durdurur: “invalid use of incomplete typedef ‘Yigin’”

Aynı main.c iki gerçekleştirimle derlenir; kullanıcı kodu struct'ın alanlarını göremez

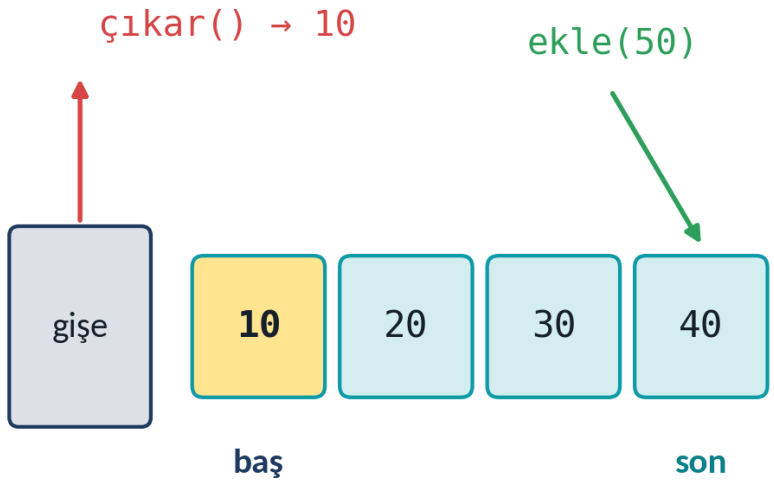
Yığın ve kuyruk: hangi uçtan girer, hangi uçtan çıkar?

Yığın (stack) · LIFO: son giren ilk çıkar



yalnızca tepeye erişilir · it, çıkar, tepe: O(1)

Kuyruk (queue) · FIFO: ilk giren ilk çıkar



aynı dört değer, aynı ekleme sırası:
yığın 40'ı, kuyruk 10'u verir

baştan çıkar, sona ekle · ikisi de O(1)

Aynı dört değer, aynı ekleme sırası: yığın 40'ı, kuyruk 10'u verir · her iki yapıda da işlemler O(1)

Üç dilde: dizi tabanlı yığında çıkar (Örnek 1)

H09_01_dizi_yigin.c — C

```c
    if (yigin_bos_mu(y)) {
        return false;
    }
    y->boyut--;
    *x = y->veri[y->boyut];
    cikar_sayisi++;
    return true;
```

H09_01_dizi_yigin.cpp — C++

```cpp
    if (yigin_bos_mu(y)) {
        return false;
    }
    y.boyut--;
    x = y.veri[y.boyut];
    cikar_sayisi++;
    return true;
```

H09_01_dizi_yigin.py — Python

```python
    if yigin_bos_mu(y):
        return False, 0
    y.boyut -= 1
    cikar_sayisi += 1
    return True, y.veri[y.boyut]
```

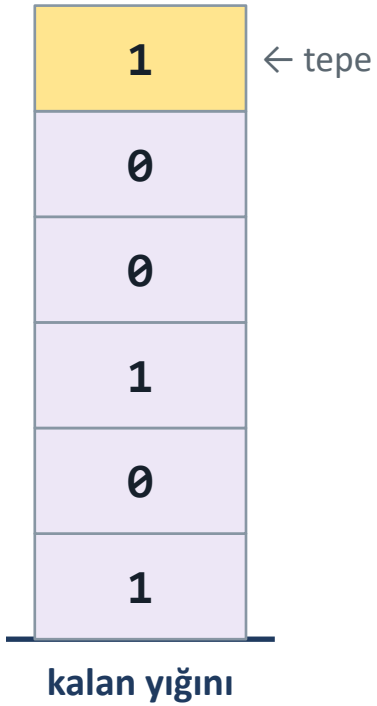
C'de bool + çıktı parametresi (`int *x`); C++'ta başvuru (`int& x`); Python'da (başarı, değer) çifti. Alt taşma üç dilde aynı sözleşmeyle bildirilir. (satırlar: C 65–71 · C++ 56–62 · Python 54–58)

TAHMİN ET

37 ikiliye çevrilirken kalanlar yığına itiliyor. Yığın boşaltılınca ne yazılır?

Bölme ve kalanlar

```
37 / 2 = 18, kalan 1
18 / 2 = 9, kalan 0
9 / 2 = 4, kalan 1
4 / 2 = 2, kalan 0
2 / 2 = 1, kalan 0
1 / 2 = 0, kalan 1
```



A 101001

B 100101

C 110010

D 010011

ALGORİTMA KİMLİK KARTI

Yığın: it / çıkar / tepe *stack: push / pop / peek*

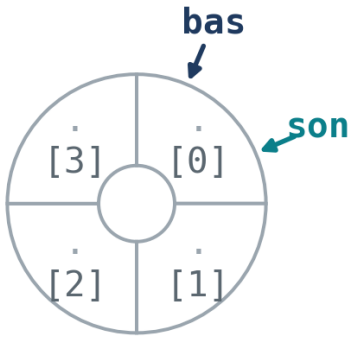
STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
LIFO: yalnızca tepeden erişim	Çıkar/tepe: boş olmamalı; it: boyut < kapasite	O(1)	O(1)	O(1)*	O(n)

Özellikler	Taşma ve alt taşma dönüş koduyla · liste tabanlıda (Örnek 2) kapasite yok, her it bir düğüm · dinamik dizide it amortize O(1)
Değişmez / doğruluk	veri[0..boyut−1] tabandan tepeye; tepe veri[boyut−1]; 0 ≤ boyut ≤ kapasite
Kütüphane karşılığı	C: yok (elle) · C++: `std::stack` (kap uyarlayıcısı; varsayılan kap `std::deque`); `std::vector::push_back` / `pop_back` · Python: `list.append` / `list.pop()` (boşta `IndexError`)
Ne zaman kullanılır	Geri al / yinele, tarayıcı geri tuşu, ifade işleme, özyinelemenin açık yığınla yazılması
Ne zaman kullanılmaz	FIFO düzen ya da ortadaki elemana erişim gerekiyorsa

Dairesel tampon: indisler dizinin sonunda başa sarar

A1

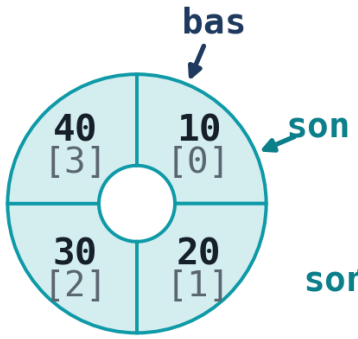
başlangıç (boş)



sayaç 0
bas = 0, son = 0

A2

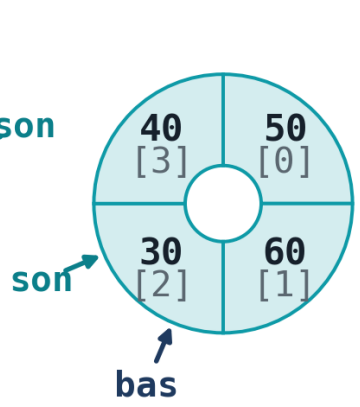
ekle 40 · sarma



sayaç 4
bas == son: dolu

A3

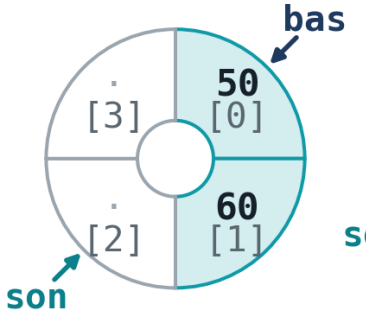
ekle 60



sayaç 4
bas == son: dolu

A4

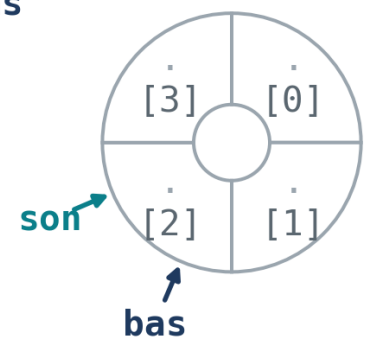
çıkar → 40 · sarma



sayaç 2
bas = 0, son = 2

A5

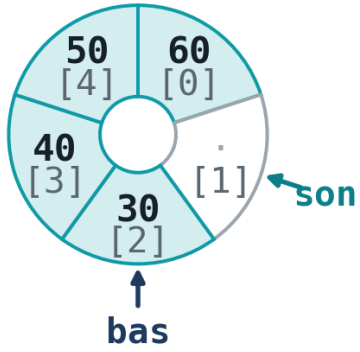
çıkar → 60



sayaç 0
bas == son: boş

B

ekle 60
(5 hücre)



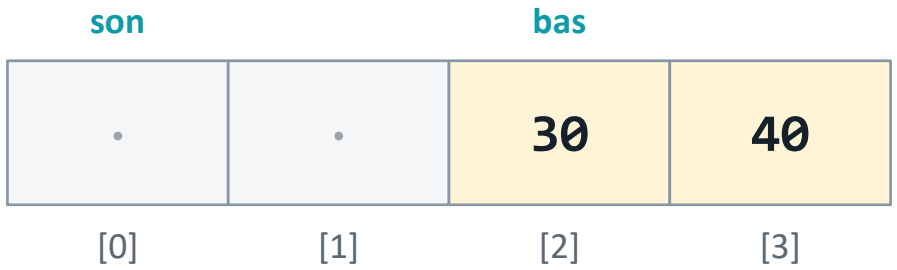
boyut 4
(son + 1) % 5 == bas:
dolu

A: sayaçlı (bas == son hem boşta hem doluda; ayrımı sayaç yapar) · B: bir boş hücreli ((son + 1) % hücre == bas ise dolu)

TAHMİN ET

bas = 2, son = 0, sayaç = 2 iken 50 eklenir. Nereye yazılır, son ne olur?

A kuyruğu (kapasite 4, sayaçlı)



Kuyruk sırası: 30 40 (baştan sona)
sayaç = 2: ne boş ne dolu
son önceki eklemede (40) 3 → 0 sarmıştı

A. hücre 2'ye; son = 3

B. hücre 4'e: dizi büyütülür

C. reddedilir, çünkü bas ≠ son

D. hücre 0'a; son = 1

Kuyruk: ekle sona, çıkar baştan — ikisi de O(1)

```
algoritma EKLE(q, x)
    eğer q.sayaç = kap ise döndür YANLIŞ
    q.veri[q.son] ← x
    q.son ← (q.son + 1) mod kap
    q.sayaç ← q.sayaç + 1;  döndür DOĞRU
algoritma ÇIKAR(q)
    eğer q.sayaç = 0 ise döndür YANLIŞ
    x ← q.veri[q.bas]
    q.bas ← (q.bas + 1) mod kap
    q.sayaç ← q.sayaç – 1;  döndür (DOĞRU, x)
```

c/H09_08_dairesel_kuyruk.c · satır 86–96 — C

```c
bool kuyruk_cikar(Kuyruk *q, int *x)
{
    if (kuyruk_bos_mu(q)) {
        return false;
    }
    *x = q->veri[q->bas];
    q->bas = ilerle(q, q->bas);
    q->sayac--;
    cikar_sayisi++;
    return true;
}
```

Dolu kuyrukta ekle: taşma; boş kuyrukta çıkar: alt taşma · değişmez: sıra veri[bas], veri[(bas + 1) % kap], …

Üç dilde: dairesel tampona ekleme

H09_08_dairesel_kuyruk.c — C

```c
    if (kuyruk_dolu_mu(q)) {
        return false;
    }
    q->veri[q->son] = x;
    q->son = ilerle(q, q->son);
    q->sayac++;
    ekle_sayisi++;
    return true;
```

H09_08_dairesel_kuyruk.cpp — C++

```cpp
    if (kuyruk_dolu_mu(q)) {
        return false;
    }
    q.veri[q.son] = x;
    q.son = ilerle(q, q.son);
    q.sayac++;
    ekle_sayisi++;
    return true;
```

H09_08_dairesel_kuyruk.py — Python

```python
    if kuyruk_dolu_mu(q):
        return False
    q.veri[q.son] = x
    q.son = ilerle(q, q.son)
    q.sayac += 1
    ekle_sayisi += 1
    return True
```

son ilerle() ile sarar; kapasite k 2'nin kuvvetiyse i % k = i & (k − 1): `13 % 8 = 5, 13 & 7 = 5; 13 % 6 = 1, 13 & 5 = 5` (Kavram 5)
(satırlar: C 75–82 · C++ 68–75 · Python 58–64)

ALGORİTMA KİMLİK KARTI

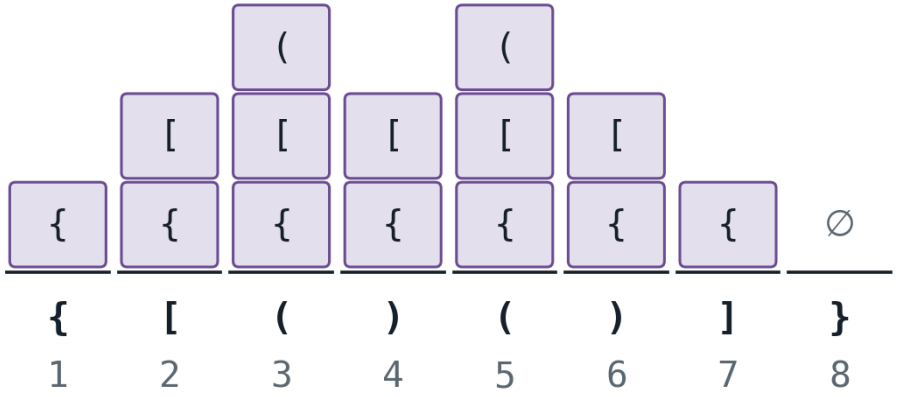
Kuyruk (dairesel tampon) *circular buffer queue; ring buffer*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
FIFO; indisler mod kapasite ile sarar	Sabit kapasite; sayaç ya da bir boş hücre	O(1)	O(1)	O(1)	O(kapasite)

Özellikler	bas == son hem boşta hem doluda: ayrımı sayaç (A) ya da kapasite + 1 hücre (B) yapar · kapasite k 2'nin kuvvetiyse mod yerine maske: i & (k − 1)
Değişmez / doğruluk	sıra veri[bas], …, veri[(bas + sayaç − 1) % kap]; 0 ≤ sayaç ≤ kap
Kütüphane karşılığı	C: yok (elle) · C++: `std::queue` (varsayılan kap `std::deque`; sabit kapasiteli halka yok) · Python: `collections.deque` (`maxlen` verilirse dolunca öbür uçtan atar); iş parçacığı güvenli `queue.Queue`
Ne zaman kullanılır	Üst sınırı bilinen akış tamponları (klavye, ses, ağ kartı halkaları); benzetimde bekleme sırası; katman katman işleme
Ne zaman kullanılmaz	Üst sınır bilinmiyorsa (liste tabanlı kuyruk, Örnek 9); ortadan silme gerekiyorsa

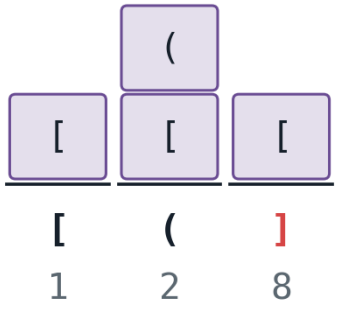
Parantez denetimi: açılış itilir, kapanış eşini çıkarır

{[()()]}



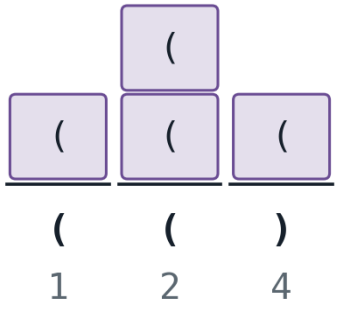
dengeli

[(a + b])



HATA sütun 8: tür uyuşmazlığı ']' geldi, ')' bekleniyordu (açılış sütun 2)

((a)



HATA satır sonu: açık kalan 1 parantez; en içteki '(' (sütun 1)

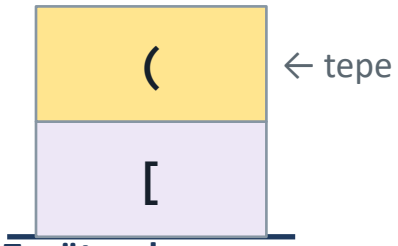
Üç hata türü: eşleşmeyen kapanış (yığın boş), tür uyuşmazlığı, açık kalan (sonda yığın dolu)

TAHMİN ET

Denetçi `[(a + b])` için ne bildirir?

sütun: karakterlerin altında





7. sütundan sonra yığın

A dengeli

B HATA sütun 9: açık kalan '['

C HATA sütun 8: ']' bulundu, ')' bekleniyordu

D HATA sütun 8: eşleşmeyen kapanış (yığın boş)

SÖZDE KOD

Parantez denetimi: tek geçiş, Θ(n)

```
1  algoritma PARANTEZ-DENETLE(s)
2      her i için (s'nin karakterleri, soldan sağa)
3          eğer s[i] açılış ise Y'ye (s[i], i) it
4          değilse eğer s[i] kapanış ise
5              eğer Y boş ise döndür HATA(eşleşmeyen, i)
6              (a, j) ← Y'den çıkar
7              eğer a, s[i]'nin eşi değil ise
8                  döndür HATA(tür uyuşmazlığı, i, j)
9      eğer Y boş değil ise döndür HATA(açık kalan)
10     döndür DENGELİ
```

- **Değişmez:** Y, okunan önekteki kapanmamış açılışları açılış sırasıyla tutar; tepede en içteki.
- Her açılış bir kez itilir, en çok bir kez çıkarılır: **Θ(n)**, ek bellek O(derinlik).
- Açılışın **sütunu** da itilir: hata iletisi iki konumu gösterir.
- Dizgi ve yorumlardaki ayraçlar ayıklanmadan yanlış hata (varyant).

ALGORİTMA KİMLİK KARTI

Parantez denetimi *balanced brackets check*

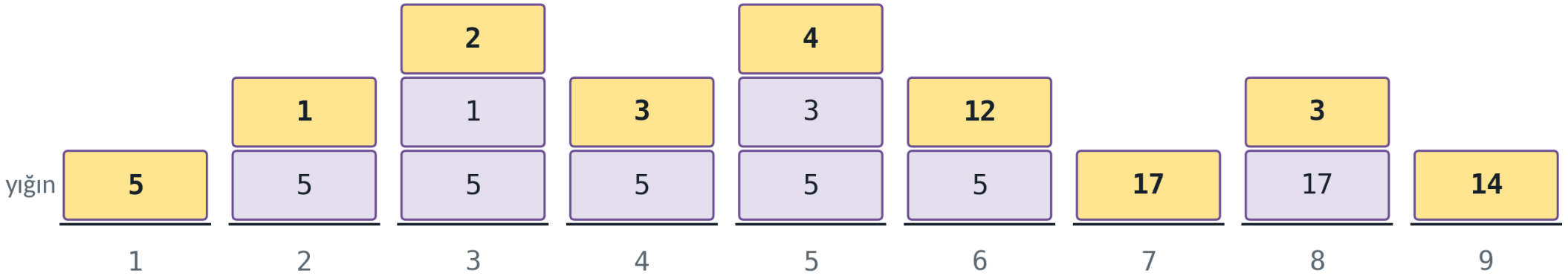
STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
Açılış it; kapanışta çıkar ve türü karşılaştır	Yok; ayraç dışı karakterler atlanır	O(1)	Θ(n)	Θ(n)	O(n)

Özellikler	Üç hata türü ve ilk hatanın sütunu · erken çıkış · tek türde sayaç yeter, türler karışınca yığın gerekir
Değişmez / doğruluk	Yığın kapanmamış açılışları açılış sırasıyla tutar; tepe en içteki
Kütüphane karşılığı	C: yok · C++: yok (`std::stack<char>` ile kurulur) · Python: yok (list yığın olarak)
Ne zaman kullanılır	Derleyici ve düzenleyicilerde ayraç eşleme; HTML/XML etiket dengesi (Kavram 8)
Ne zaman kullanılmaz	Dizgi/yorum ayıklanmadan; tam sözdizimi denetimi için tek başına yetmez

Sonek değerlendirme: işlenen yığını, soldan sağa tek geçiş

belirteç	5	1	2	+	4	*	+	3	-
	it	it	it	1 + 2	it	3 * 4	5 + 12	it	17 - 3
				a ∘ *b*		*a* ∘ *b*	*a* ∘ *b*		*a* ∘ *b*

sonuç 14



araek 5 + (1 + 2) × 4 − 3 · önek − + 5 × + 1 2 4 3 · sonek 5 1 2 + 4 × + 3 −: sonekte parantez ve öncelik kuralı gerekmez

TAHMİN ET

Sonek 2 3 ^ 2 ^ kaçtır?

belirteçler (soldan sağa)



sayı → it
işleç → önce b, sonra a çıkar; a ^ b'yi it

(boş)

işlenen yığını

A 64

B 512

C 36

D HATA: eksik işlenen

Sonek değerlendirme: önce b, sonra a

```
algoritma SONEK-DEĞERLENDİR(belirteçler)
    her t için (soldan sağa)
        eğer t sayı ise Y'ye t'yi it
        değilse eğer t ikili işleç ise
            eğer Y'de 2'den az değer ise döndür HATA(eksik)
            b ← Y'den çıkar;  a ← Y'den çıkar
            Y'ye (a t b)'yi it   ▷ 0'a bölme, taşma: HATA
    eğer Y'de tam 1 değer değil ise döndür HATA(fazla)
    döndür Y'nin tepesi
```

- **Değişmez:** Y, okunan önekteki tamamlanmış alt ifadelerin değerlerini soldan sağa tutar.
- İlk çıkan **sağ** işlenendir: 10 3 − = 7; sıra ters alınırsa −7 (Hata Avcısı 4).
- + ve × değişmeli olduğu için ters sıra testte **gizlenir**: test − / ^ içermeli.
- Bölme sıfıra doğru keser, üç dilde aynı: `7 2 ~ / = -3 [it 4, çıkar 3, derinlik 2]`.

C: sonekte ikili işleç (Örnek 4)

c/H09_04_sonek_degerlendirme.c · satır 182–190

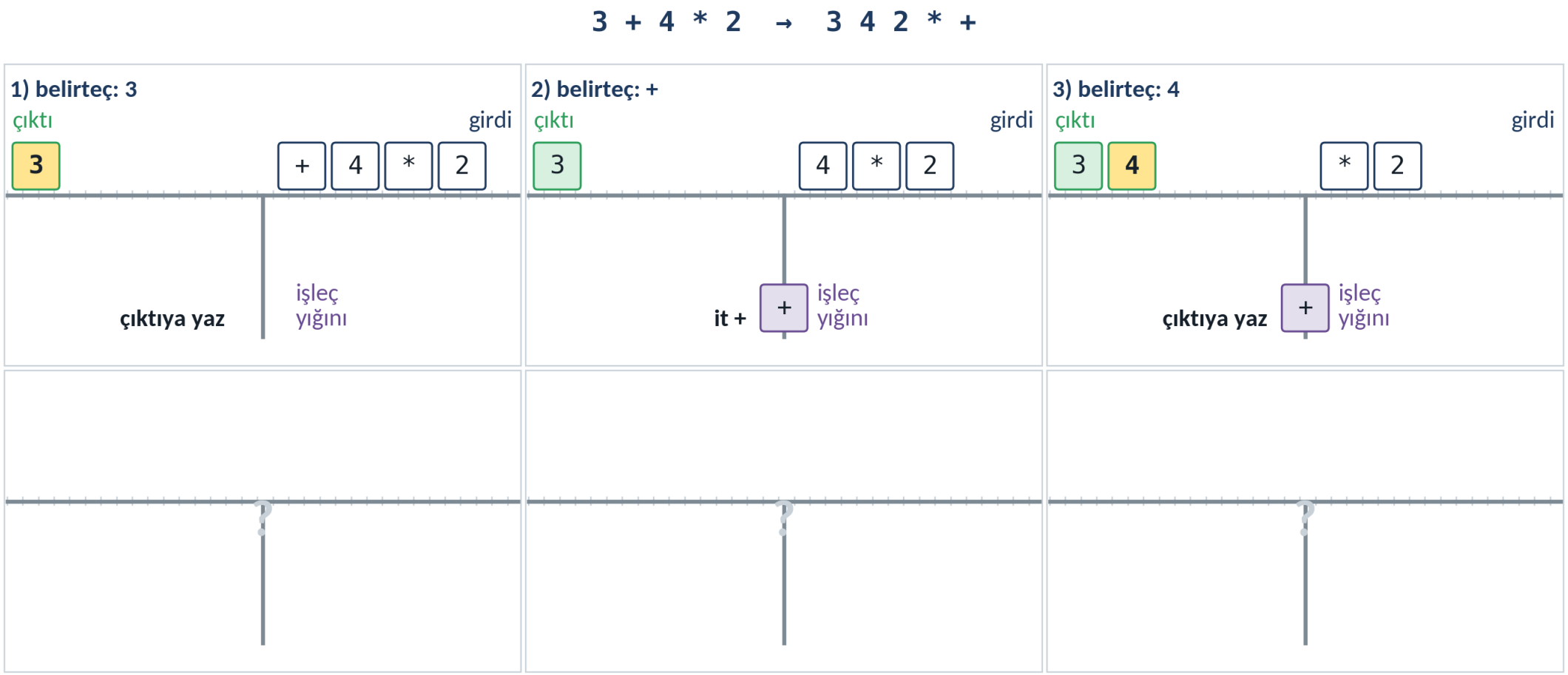
C

```c
        } else if (uz == 1 && strchr("+-*/^", c) != NULL) {
            if (!yigin_cikar(&y, &b) || !yigin_cikar(&y, &a)) {
                hata = EKSIK_ISLENEN;  /* önce b (tepe), sonra a */
            } else {
                hata = islem_uygula(c, a, b, &r);
                if (hata == HATA_YOK) {
                    yigin_it(&y, r);
                }
            }
```

10 3 - (ÖRNEK 4'ÜN İZİ)

```
10 3 -
  10          -> yığın: [10]
  3           -> yığın: [10 3]
  -           -> yığın: [7]
  sonuç: = 7   [it 3, çıkar 2, derinlik 2]
```

Manevra sahası: işlenen çıktıya, işleç yan hatta bekler



Sağdan girdi rayı, soldan çıktı rayı, ortada kör yan hat (işleç yığını): yalnızca en öndeki vagon hareket edebilir

TAHMİN ET

3 + 4 * 2: sıradaki belirteç *. Ne olur?

çıktı

3	4

kalan girdi

*	2

öncelik
+ − : 1
* / : 2
^ : 4 (sağdan)



işleç yığını

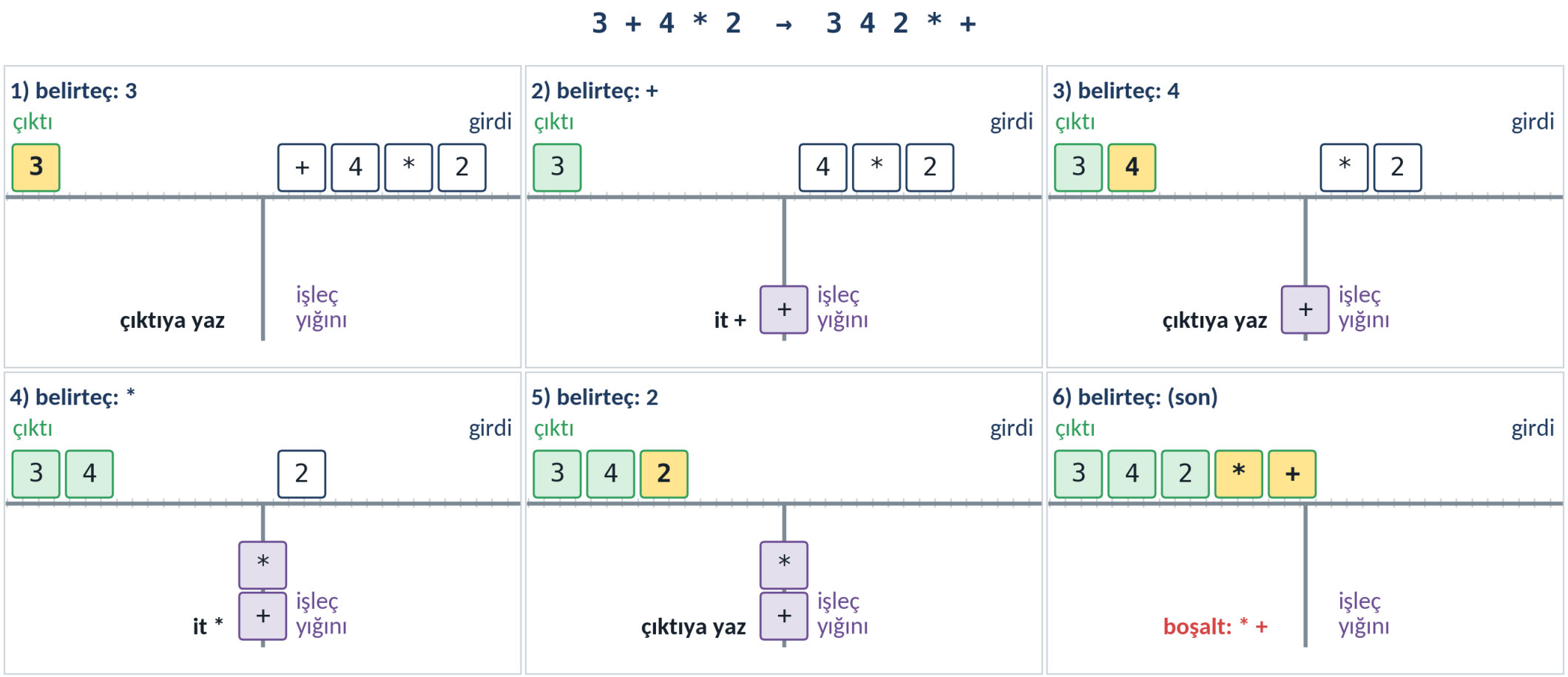
A + önce çıktıya, sonra * itilir

B * itilir: yığın [+ *]

C * doğrudan çıktıya yazılır

D + ve * birlikte çıktıya: 3 4 + *

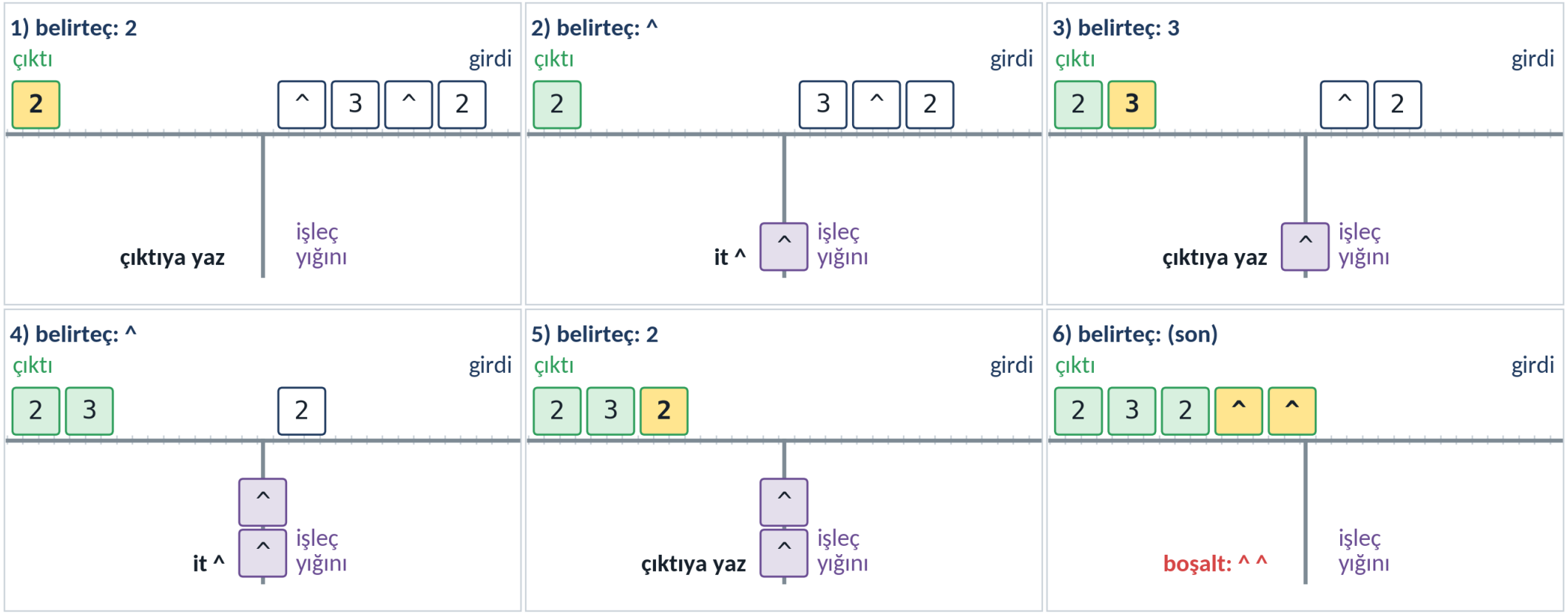
3 + 4 * 2 → 3 4 2 * +: yığın sırayı tersine çevirir



*Altı adım · 3 + 4 * 2 -> 3 4 2 * + [belirteç 5, it 2, çıkar 2, derinlik 2]*

Birleşme yönü: 2 ^ 3 ^ 2 = 2 ^ (3 ^ 2)

2 ^ 3 ^ 2 → 2 3 2 ^ ^



Eşit öncelikte ^ tepedekini çıkarmaz, üstüne itilir · 2 ^ 3 ^ 2 -> 2 3 2 ^ ^ [belirteç 5, it 2, çıkar 2, derinlik 2] = 512

Shunting-yard: dört kural

```
algoritma SHUNTING-YARD(belirteçler)
    her t için (soldan sağa)
        eğer t işlenen ise Ç'ye t'yi ekle
        değilse eğer t = ( ya da t = ~ ise Y'ye t'yi it
        değilse eğer t = ) ise
            iken tepe ≠ ( ise Ç'ye Y'den çıkar
            ( yoksa HATA; varsa ( atılır
        değilse   ▷ t işleç
            iken tepe işleç ve ÖNCE-ÇIKAR(tepe, t)
                Ç'ye Y'den çıkar
            Y'ye t'yi it
    iken Y boş değil: ( ise HATA; değilse Ç'ye çıkar
```

- **ÖNCE-ÇIKAR(ü, t):** öncelik(ü) > öncelik(t) **ya da** (eşit **ve** t soldan birleşmeli).
- **Değişmez:** Ç, okunan önekteki tamamlanmış alt ifadelerin sonekidir.
- Her işleç bir kez itilir, en çok bir kez çıkarılır: **Θ(n)**.
- Tekli eksi (~, öncelik 3, sağdan) başta, (ya da işleçten sonra gelen –'dir.

C: ikili işleç dalı (Örnek 5)

c/H09_05_shunting_yard.c · satır 240–249

C

```c
            while (yigin_tepe(&y, &j) && b[j].tur == ISLEC &&
                   once_cikar_mi(b[j].islec, t.islec)) {
                yigin_cikar(&y, &j);
                cikti[(*k)++] = j;
                strcat(eylem, eylem[0] == '\0' ? "çıkar: " : " ");
                belirtec_ekle(eylem, s, &b[j]);
            }
            strcat(eylem, eylem[0] == '\0' ? "it " : "; it ");
            belirtec_ekle(eylem, s, &t);
            yigin_it(&y, i);
```

ÖRNEK 5'İN İZİ: İLK DÖRT BELİRTEÇ

belirteç	işleç yığını	çıktı	eylem
a	[]	a	çıktıya yaz
-	[-]	a	it -
b	[-]	a b	çıktıya yaz
-	[-]	a b -	çıkar: -; it -

ALGORİTMA KİMLİK KARTI

Shunting-yard: araek → sonek *shunting-yard algorithm; infix to postfix*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
İşleç yığını; (bariyer,) boşaltır	Öncelik ve birleşme tablosu: + − 1; * / 2; ~ 3 sağdan; ^ 4 sağdan	Θ(n)	Θ(n)	Θ(n)	O(n)

Özellikler	2 ^ 3 ^ 2 -> 2 3 2 ^ ^ (sağdan) · a - b - c -> a b - c - (soldan) · hatalar sütunuyla
Değişmez / doğruluk	Çıktı, okunan önekteki tamamlanmış alt ifadelerin sonekidir
Kütüphane karşılığı	C: yok · C++: yok · Python: yok (`ast`/`eval` algoritmayı göstermez)
Ne zaman kullanılır	Hesap makinesi ve yorumlayıcılarda araek ifade: sonek çıktı işlenen yığınıyla tek geçişte değerlendirilir (Örnek 4, 6)
Ne zaman kullanılmaz	Fonksiyon çağrısı, indis, atama gibi yapılar eklenmeden; bütün bir dil için özyinelemeli iniş

Geri al / yinele: iki yığın

komut	belge	geri yığını (taban → tepe)	yinele yığını
`yaz Merhaba`	"Merhaba"	**+Merhaba**	
`yaz ␣dunya`	"Merhaba dunya"	+Merhaba | **+␣dunya**	
`sil 5`	"Merhaba "	+Merhaba | +␣dunya | **−dunya**	
`yaz ␣yigin`	"Merhaba yigin"	+Merhaba | +␣dunya | −dunya | **+␣yigin**	
`geri`	"Merhaba "	+Merhaba | +␣dunya | **−dunya**	**+␣yigin**
`geri`	"Merhaba dunya"	+Merhaba | **+␣dunya**	+␣yigin | **−dunya**
`yinele`	"Merhaba "	+Merhaba | +␣dunya | **−dunya**	**+␣yigin**
`yaz !`	"Merhaba !"	+Merhaba | +␣dunya | −dunya | **+!**	**boşaltıldı (yeni işlem)**
`yinele`	"Merhaba !"	+Merhaba | +␣dunya | −dunya | **+!**	*(yinelenecek işlem yok)*

Her işlem tersiyle birlikte geri yığınına itilir · geri al: geriden çıkar, tersini uygula, yineleye it · yeni işlem yinele yığınını boşaltır

Sonraki büyük eleman: cevabı bulunmayanlar yığında bekler

i	a[i]	dizi (mor: yığında, yeşil: cevaplandı)	çıkarılanlar → cevap	yığın (taban → tepe)
0	3	**3** 8 4 1 2 6 5 9 7	–	3
1	8	3 **8** 4 1 2 6 5 9 7	3 → 8	8
2	4	3 8 **4** 1 2 6 5 9 7	–	8 4
3	1	3 8 4 **1** 2 6 5 9 7	–	8 4 1
4	2	3 8 4 1 **2** 6 5 9 7	1 → 2	8 4 2
5	6	3 8 4 1 2 **6** 5 9 7	2 4 → 6	8 6
6	5	3 8 4 1 2 6 **5** 9 7	–	8 6 5
7	9	3 8 4 1 2 6 5 **9** 7	5 6 8 → 9	9
8	7	3 8 4 1 2 6 5 9 **7**	–	9 7
	sonraki büyük	**8 9 6 2 6 9 9 -1 -1**	it = 9, çıkar = 9 (her biri ≤ n = 9)	

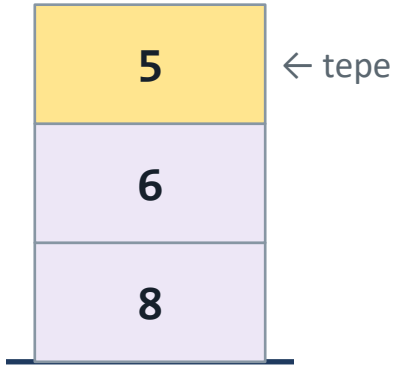
a = [3 8 4 1 2 6 5 9 7] · yığın tabandan tepeye artmayan · sonraki büyük: 8 9 6 2 6 9 9 -1 -1

TAHMİN ET

i = 6'dan sonra yığın [8 6 5]; a[7] = 9 geliyor. Kaç eleman çıkar?

a (indisler altta)

3	8	4	1	2	6	5	9	7
0	1	2	3	4	5	6	7	8



yığın (değerler)

A 1 eleman: yalnızca 5

B 2 eleman: 5 ve 6

C 3 eleman: 5, 6, 8

D hiçbiri: 9 itilir

ALGORİTMA KİMLİK KARTI

Sonraki büyük eleman (monotonik yığın) *next greater element; monotonic stack*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
Bekleyen indisler yığında; yeni eleman küçük tepeleri çıkarır	Yok	Θ(n)	Θ(n)	Θ(n)	O(n)

Özellikler	Her indis bir kez itilir, en çok bir kez çıkarılır: it + çıkar ≤ 2n · azalan girdide (n = 1 000) yığınla 999, kaba kuvvetle 499 500 karşılaştırma
Değişmez / doğruluk	Değerler tabandan tepeye artmayan; hiçbirinin sonraki büyüğü henüz görülmedi
Kütüphane karşılığı	C: yok · C++: yok · Python: yok
Ne zaman kullanılır	Sonraki/önceki büyük-küçük; hisse açıklığı; histogramdaki en büyük dikdörtgen (Merak)
Ne zaman kullanılmaz	Soru bir aralığın en büyüğüyse (kayan pencere: iki uçlu kuyruk)

Kayan pencere en büyüğü: monotonik iki uçlu kuyruk

i	a[i]	dizi (çerçeve: pencere, k = 3)	deque (ön → arka)	en büyük	atılanlar
0	**4**	**4** 2 12 3 8 1 5 7 6 2	**4**	–	–
1	**2**	4 **2** 12 3 8 1 5 7 6 2	**4** 2	–	–
2	**12**	4 2 **12** 3 8 1 5 7 6 2	**12**	12	arkadan 2, arkadan 4
3	**3**	4 2 12 **3** 8 1 5 7 6 2	**12** 3	12	–
4	**8**	4 2 12 3 **8** 1 5 7 6 2	**12** 8	12	arkadan 3
5	**1**	4 2 12 3 8 **1** 5 7 6 2	**8** 1	8	önden 12
6	**5**	4 2 12 3 8 1 **5** 7 6 2	**8** 5	8	arkadan 1
7	**7**	4 2 12 3 8 1 5 **7** 6 2	**7**	7	önden 8, arkadan 5
8	**6**	4 2 12 3 8 1 5 7 **6** 2	**7** 6	7	–
9	**2**	4 2 12 3 8 1 5 7 6 **2**	**7** 6 2	7	–

a = [4 2 12 3 8 1 5 7 6 2], k = 3 · arkadan: küçük-eşitler atılır · önden: pencereden çıkan indis · pencere en büyükleri: 12 12 12 8 8 7 7 7

TAHMİN ET

k = 3, i = 4'ten sonra deque [12 8]; a[5] = 1. Pencerenin en büyüğü?

a (pencere mavi) pencere 3..5

4	2	12	3	8	1	5	7	6	2
0	1	2	3	4	5	6	7	8	9

deque (önden arkaya, değerler)

12	8

ön (indis 2) arka (indis 4)

A. 8: önden 12 atılır (indis 2 < 3), 1 arkaya eklenir

B. 12: 12 hâlâ deque'nun önünde

C. 1: yeni gelen her zaman en büyük

D. 8: ama 12 arkadan atılır

ALGORİTMA KİMLİK KARTI

Kayan pencere en büyüğü (deque) *sliding window maximum; monotonic deque*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
İndis deque'su; arkadan küçük-eşitler, önden pencere dışı atılır	$1 \le k \le n$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$O(k)$

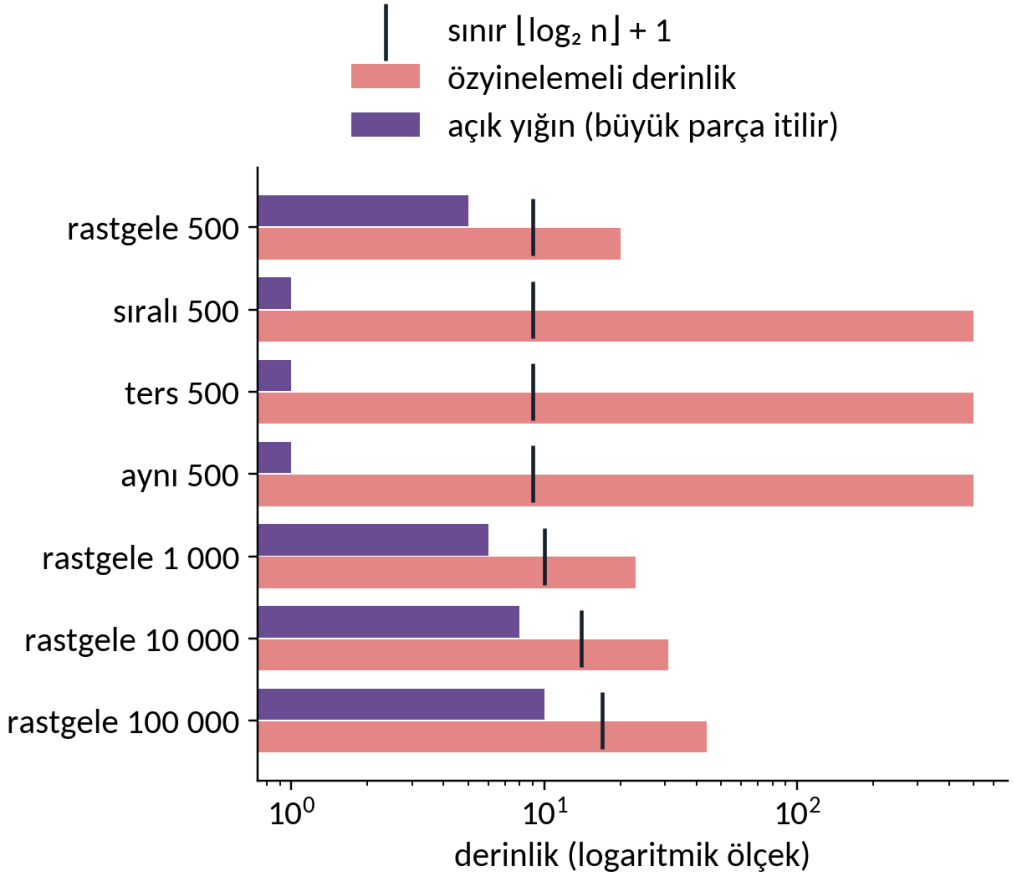
Özellikler	Her indis bir kez girer, en çok bir kez çıkar: Θ(n), k'dan bağımsız · kaba kuvvet $(n - k + 1)(k - 1)$ karşılaştırma
Değişmez / doğruluk	İndisler artan, değerler kesin azalan, hepsi pencerede; ön = pencerenin en büyüğü
Kütüphane karşılığı	C: yok · C++: `std::deque` (iki uçlu kuyruk kabı) · Python: `collections.deque` (append, pop, popleft)
Ne zaman kullanılır	Akan veride son k ölçümün en büyüğü/küçüğü; en büyük − en küçük ≤ d (Alıştırma 9.13)
Ne zaman kullanılmaz	Pencere işlemi ters çevrilebiliyorsa (toplam: önek toplamı yeter)

Özyinelemeden açık yığına: büyük parçayı it, küçüğü hemen işle

a = [7 3 5 9 2 4 1 8 6]: açık yığının içeriği

çıkar 0..8	0..4 6..7	derinlik 2
çıkar 6..7	0..4	derinlik 1
çıkar 0..4	1..4	derinlik 1
çıkar 1..4	3..4	derinlik 1
çıkar 3..4	*(boş)*	derinlik 0

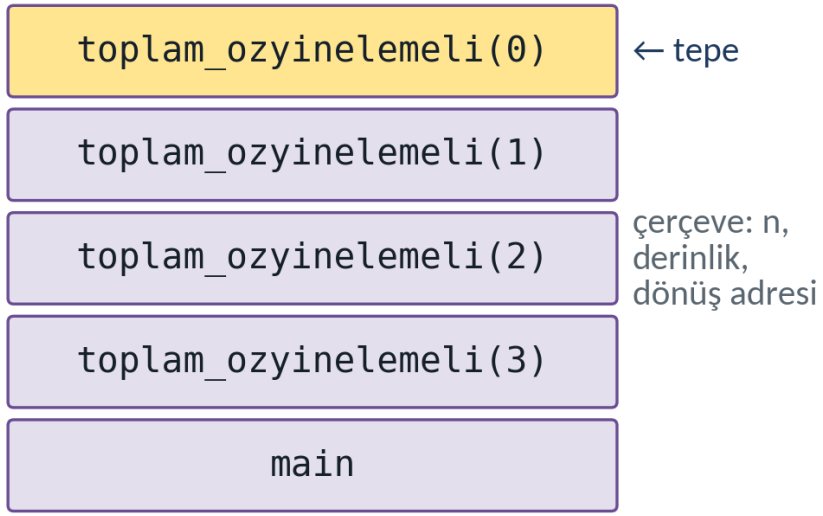
en büyük derinlik 2 (sınır $\lfloor \log_2 n \rfloor + 1 = 4$); büyük parça itilir



Derinlik ≤ ⌊log₂ n⌋ + 1 · n = 10⁵ rastgele: açık yığın 10 (sınır 17), özyinelemeli 44 · sıralı girdide özyinelemeli derinlik n

Kaputun Altında: çağrı yığını ile veri yapısı olarak yığın

örtük: çağrı yığını (her çağrı bir çerçeve)



C (-O0): n = 1000 → 1001 çağrı, çalışır
n = 10 000 000 → SIGSEGV (çağrı yığını taşar)

Python: sınır 1000 → RecursionError

açık: öbekte ayrılan dizi (malloc)



çağrı yerine it: yigin[boyut++] = k
dönüş yerine çıkar: yigin[--boyut]

n = 10 000 000: toplam = 50 000 005 000 000
en büyük yığın = 10 000 000 eleman: çökmez

sınır: öbek belleği (çağrı yığını değil)

Solda örtük yığın: her çağrı bir çerçeve, derinlik sınırlı (C: SIGSEGV, Python: RecursionError) · sağda öbekte açık yığın (malloc)

Kaputun Altında: CPython bayt kodu bir yığın makinesinde çalışır

kavram/H09_K02_dis_bayt_kodu.py · satır 44–53

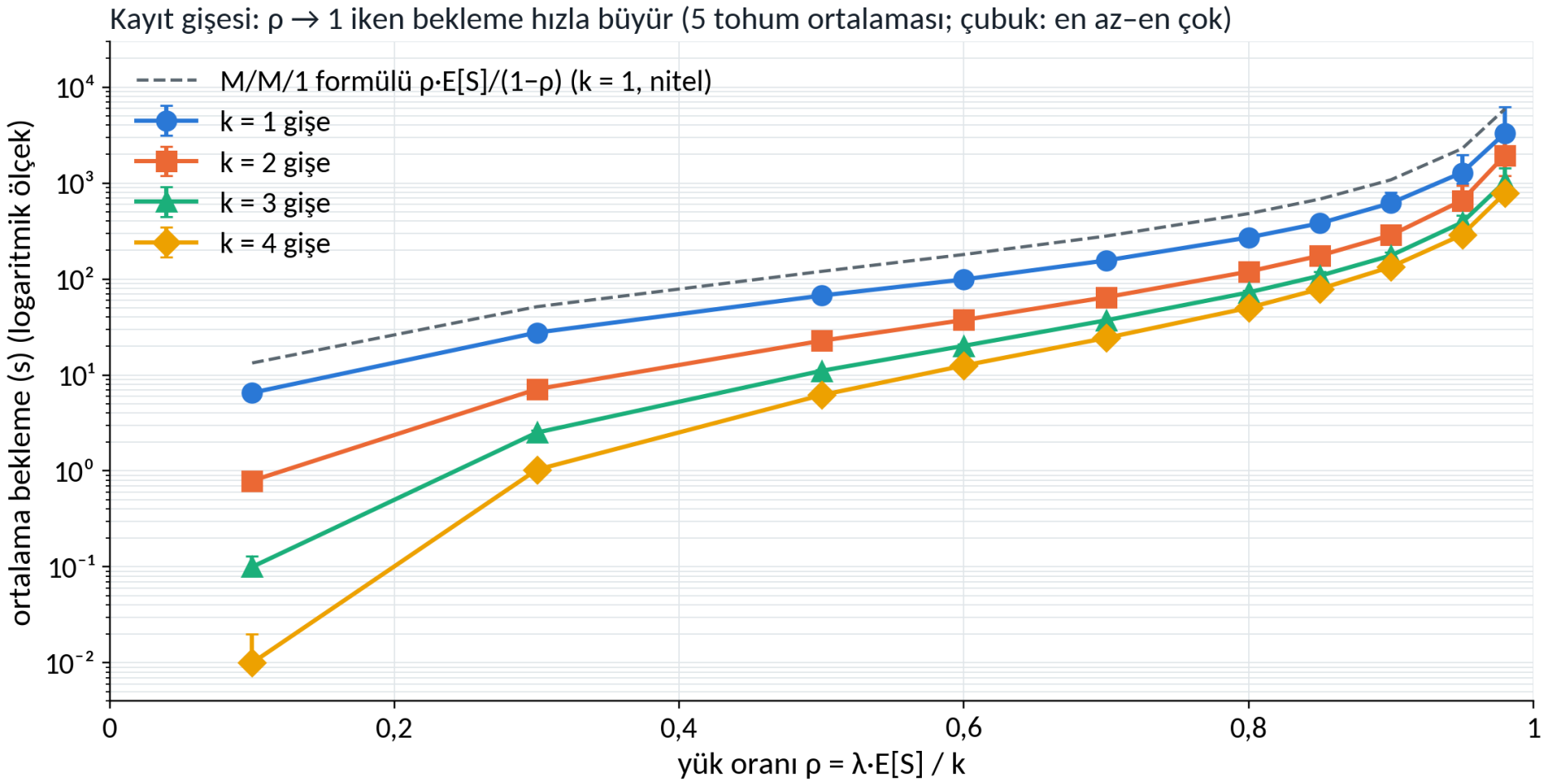
Python

```
44      print("\n1) def f(a, b, c): return a + b * c")
45      dis.dis(f)
46      print("\n2) İşlenen yığını adım adım (önce b * c, sonra a + ...)")
47      yigin_izle(f)
48      print("\n3) Sabit katlama: x = 3 + 4 * 2")
49      kod = compile("x = 3 + 4 * 2", "<ifade>", "exec")
50      dis.dis(kod)
51      sabitler = [c for c in kod.co_consts if isinstance(c, int)]
52      print(f"  co_consts içindeki tamsayılar: {sabitler} "
53            "(3, 4 ve 2 ayrı ayrı saklanmaz)")
```

BAYT KODU VE SABİT KATLAMA (PYTHON 3.11.15)

```
 15           2 LOAD_FAST                0 (a)
              4 LOAD_FAST                1 (b)
              6 LOAD_FAST                2 (c)
              8 BINARY_OP                5 (*)
             12 BINARY_OP                0 (+)
             16 RETURN_VALUE
  co_consts içindeki tamsayılar: [11] (3, 4 ve 2 ayrı ayrı saklanmaz)
```

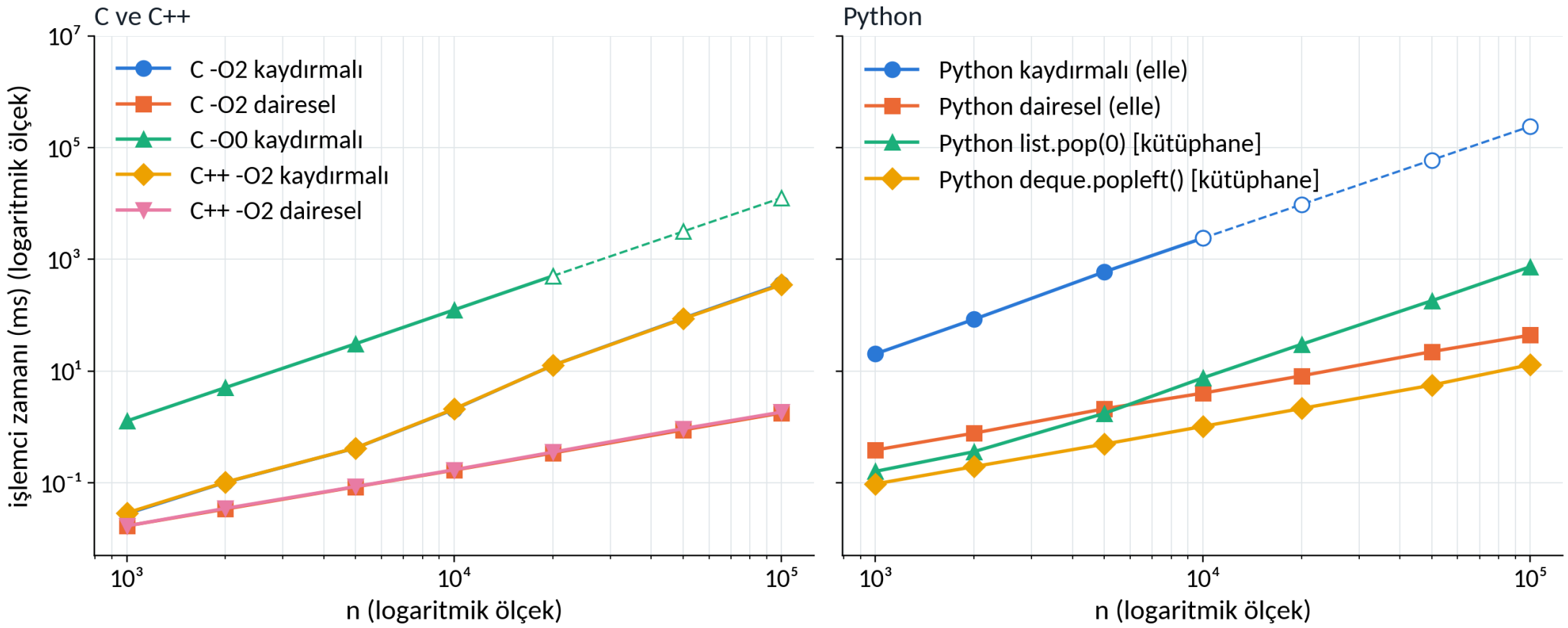
Haftanın Deneyi: ρ → 1 iken bekleme patlar



Örnek 10'un benzetimi, 5 tohum ortalaması · tek gişe: ρ = 0,5'te 67,20 s, ρ = 0,9'da 618,00 s · iki gişe ρ = 0,9'da 286,42 s

Haftanın Deneyi: kaydırmalı kuyruk Θ(n²), dairesel Θ(n)

n ekle + n çıkar (içi boş işaret + kesikli çizgi: n^2 modeliyle kestirim, ölçülmedi)



n = 10⁵: C -O2 kaydırmalı / dairesel = 204 kat · Python list.pop(0) / deque.popleft() = 56 kat · kesikli: n² modeliyle kestirim

Hata Avcısı: altı hata, gerçek belirtiler

#	Hata	Belirti (bu makinede)	Çözüm
1	Boş yığından çıkarma	boyut = -1; ASan: heap-buffer-overflow	önce yigin_bos_mu
2	Dolu/boş ayrımı yalnız bas == son	4 eleman eklendi; çıkarılanlar: (hiçbiri)	sayaç ya da bir boş hücre
3	^ işlecini soldan birleştirmek	2 ^ 3 ^ 2 -> sonek 2 3 ^ 2 ^ = 64	eşitlikte yalnız soldan birleşmeliyse çıkar
4	Sonekte işlenen sırasını ters almak	10 3 - = -7 (beklenen 7) <-- YANLIŞ	önce b, sonra a; testte – / ^
5	Python'da list.pop(0) ile kuyruk	n = 10 000: 7,88 ms · n = 80 000: 515,96 ms	deque.popleft()
6	Dizi yığında kapasite aşımı	10 eleman itildi (kapasite 4); boyut = 10; ASan: heap-buffer-overflow H09_HA6_kapasite_asimi.c:24 in yigin_it	taşma denetimi ya da iki katına büyüt

Hata Avcısı 3 yakından: ^ soldan birleşirse

Hatalı: eşitlikte her zaman çıkar (>=)

hata_avcisi/H09_HA3_us_soldan.cpp · satır 22–25 — C++

```cpp
22  static bool once_cikar_mi(char ust, char gelen)
23  {
24      return oncelik(ust) >= oncelik(gelen);
25  }
```

DEĞER: HATALI → DÜZELTİLMİŞ		
2 ^ 3 ^ 2	64 → 512	X
2 ^ 2 ^ 3	64 → 256	X
4 ^ 3 ^ 2	4096 → 262144	X
2 * 3 ^ 2	18 → 18	
8 - 4 - 2	2 → 2	
(2 ^ 3) ^ 2	64 → 64	

Düzeltilmiş: eşitlikte birleşme yönüne bak

hata_avcisi/H09_HA3_us_soldan_duzeltilmis.cpp · satır 20–27 — C++

```cpp
20  static bool sag_birlesmeli(char op) { return op == '^'; }
21
22  // DÜZELTME: eşitlikte birleşme yönü belirleyicidir
23  static bool once_cikar_mi(char ust, char gelen)
24  {
25      return oncelik(ust) > oncelik(gelen) ||
26             (oncelik(ust) == oncelik(gelen) && !sag_birlesmeli(gelen));
27  }
```

Belirti yalnızca ^ zincirlerinde (X): 2 ^ 3 ^ 2 hatalı sürümde 64, doğrusu 512 · soldan birleşen – iki sürümde aynı: 8 - 4 - 2 = 2

Dil Penceresi: yığın, kuyruk, kapsülleme, bölme

Öğe	C	C++	Python
Yığın	yok; elle (dizi ya da liste)	std::stack: kap uyarlayıcısı (deque)	list: append / pop()
Kuyruk	yok; dairesel tampon	std::queue: uyarlayıcı (deque)	collections.deque: popleft()
Kapsülleme	opak struct	class + private	gelenek: _alan
Hata bildirimi	dönüş kodu (bool)	dönüş kodu ya da istisna	IndexError, queue.Empty
Baştan çıkarma	kaydırma Θ(n), dairesel O(1)	queue::pop O(1)	list.pop(0) Θ(n)!
−7 / 2 ve −7 % 2	−3 ve −1 (sıfıra doğru)	C ile aynı	// ile −4 ve 1 (aşağı)

Mühendis Gözüyle: dolu tamponda ne yapılır?

reddet

ekleme false döner;
yeni veri alınmaz



kapasite 3, dolu

Örnek 1, 8, 10;
v2.8 kayıt kuyruğu

üzerine yaz

en eski atılır;
yeni veri girer



1..5 eklendikten sonra

deque(maxlen=3);
v2.8 geri al geçmişi

bekle

üretici, yer açılana
dek bekler



kapasite 3, dolu

queue.Queue(maxsize).put()
(iş parçacıkları)

Reddet (Örnek 8, 10; v2.8 kayıt kuyruğu: 1 601 ret) · en eskinin üzerine yaz (deque(maxlen=3): [3, 4, 5]) · bekle (queue.Queue.put)

İleri Bakış · Merak Kutusu

H11: graflar

- DFS bir yığınla, BFS bir kuyrukla dolaşır (Kavram 4'ün iki döngüsü).
- Tek fark bekleyenlerin tutulduğu yapı.

Biçimsel diller · kuyruk kuramı

- Bağlamdan bağımsız dilleri tanıyan makine **yığınlı özdevinir**: sonlu denetim + yığın; Shunting-yard işleç öncelikli bir ayrıştırıcıdır.
- **Kuyruk kuramı** (M/M/1, P–K) olasılık ve benzetim derslerinde sürer; işletim sistemi **zamanlayıcısı** hazır süreçleri kuyrukta tutar.

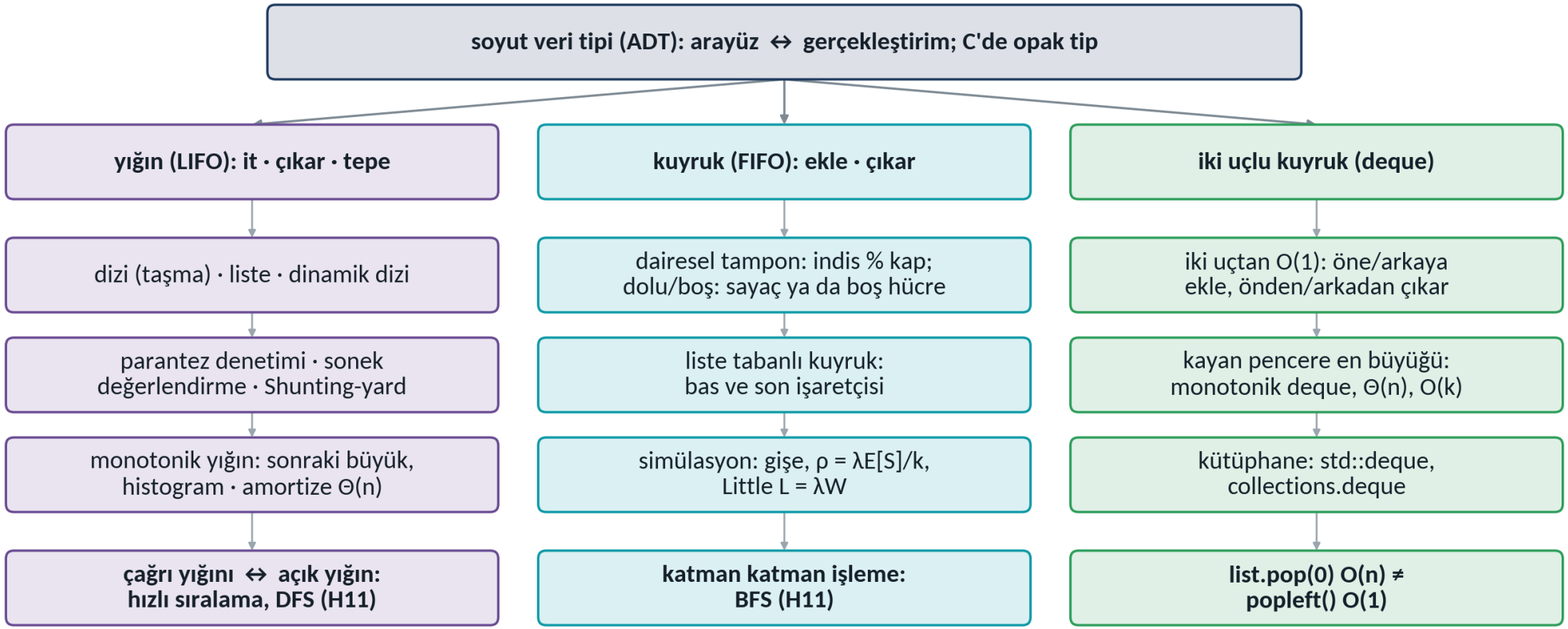
Merak: histogram (Örnek 14)

- En büyük dikdörtgen: monotonik yığınla Θ(n).
- `en büyük alan = 9 (kâhin 9)`; alan 32 biti aşabilir: long long.
- Soru: 0/1 matristeki en büyük dikdörtgen?

Mini-OBS 2 v2.8: geri al / yinele ve ders kayıt kuyruğu

- **Yazılır:** `kuyruk.c` (sayaçlı dairesel tampon, doluysa ret), `geri_al.c` (iki yığın, sınır 100, en eski kayıt düşer), `kayit_gunu.c` (kayıt günü: FIFO istek kuyruğu, gişeler, ders kontenjanları, bekleme listeleri), yeni komutlar: `geri`, `yinele`, `gecmis`.
- **Kural:** yeni işlem yinele yığınını boşaltır (ve kayıtları **serbest bırakır**); silmenin tersi için kaydın **kopyası** saklanır.
- **Kabul (seçme):** varsayılanlarla (2 gişe, kuyruk 64) `Kuyruk dolu: ret  : 1601`, ortalama bekleme 183,43 s; `--gise 4` ile ret 0; iki çalıştırma bayt bayt aynı.
- **Test:** geri al ∘ işlem = kimlik, rastgele işlem dizileriyle ve her adımda tablonun anlık görüntüsüyle (kâhin); `make asan`, `make valgrind` temiz.

Özet: Hafta 9 kavram haritası



ADT: arayüz ↔ gerçekleştirim · yığın (LIFO) · kuyruk (FIFO) · iki uçlu kuyruk · her eleman bir kez girer, en çok bir kez çıkar

Hazırlık: bir sonraki derse kadar

- **Okuma:** ders notu Bölüm 3–6 (kavramlar, algoritmalar, çözümlü örnekler, deney); Ek A'da kendi dilinizin karşılıkları; Bölüm 12'deki kendini değerlendirme listesi.
- **Alıştırmalar 9.1–9.15:** önce izleme ve çıktı tahmini (9.1–9.3), sonra hata bulma (9.8–9.9) ve en az bir kod yazma (9.11–9.13).
- **Atölyeler (Bölüm 11.8–11.10):** ray manevra oyunu (Shunting-yard kartlarla), mini derleyici adımı (eşli: dört işlem, parantez, üs), kuyruk simülasyonu (gişe sayısı ve bekleme).
- **Web:** dört oynatıcı (yığın/kuyruk, adım adım Shunting-yard, sonek hesap makinesi, gişe benzetimi) ve 10 soruluk quiz.
- **Mini-OBS 2 v2.8:** `kuyruk.c`, `geri_al.c`, `kayit_gunu.c`; kabul ölçütleri v2.8 belgesi § 10.
- **H10'a hazırlık (Karma Tabloları):** anahtarı aramadan, neredeyse tek adımda bulmak mümkün mü? Çakışan anahtarlar H8'in bağlı listeleriyle zincirlenecek.

ÇIKIŞ BİLETİ

Dersten çıkmadan önce üç soru

1 Kapasitesi 4 olan sayaçlı dairesel tamponda bas = 3, sayaç = 2. Sıradaki eleman hangi indise yazılır?

2 `(1 + 2) * 3 - 4`'ü Shunting-yard ile soneke çevirin. İşleç yığını en çok kaç derinliğe çıkar?

3 Sonraki büyük eleman algoritması neden $O(n^2)$ değil $O(n)$'dir? Bir cümleyle.